

ShopMate AI — Intelligent Retail Recommendation System'

A Project report submitted for the partial fulfilment of the requirements to
award the degree of
Bachelor of Technology
in
Computer Science and Engineering
School of Engineering and Sciences

Submitted by

Ramakriahna Eswar Naidu	AP23110010116
Bommidi Pranay	AP23110010149
Valluri Sai Raju	AP23110010153
Duddela Sooraj Krishna	AP23110010122
Karnati Sasank	AP23110010092



Submitted to
Konakala Rithvik

SRM UNIVERSITY - AP, ANDHRA PRADESH

1. PROJECT OVERVIEW

ShopMate AI is an advanced intelligent retail recommendation system designed to transform the way users discover products online. Unlike traditional e-commerce platforms that rely on static keyword-based search, ShopMate AI leverages modern AI technologies to understand user intent, personalize recommendations, and deliver real-time, context-aware product suggestions.

At its core, the system integrates **Google Gemini 2.5 Flash**, **Firecrawl API**, and a **Memento-style persistent memory engine** to create a seamless and intelligent shopping assistant experience. Users can simply enter natural-language queries such as *“wireless headphones under 5000”*, and the system interprets the request by extracting key parameters like product category and budget.

One of the standout features of ShopMate AI is its **continuous learning capability**. The system captures meaningful signals from each user interaction—such as preferences, spending patterns, and frequently searched categories—and stores them persistently across sessions. This eliminates the need for explicit user profiles or sign-ups while still enabling deep personalization. Over time, the system evolves into a highly tailored recommendation engine unique to each user.

The platform operates through a **four-stage intelligent pipeline**:

1. **Memory Layer** – Extracts and stores user preferences (category, budget, history)
2. **Crawler Layer** – Retrieves real-time product data from platforms like Amazon and Flipkart using Firecrawl
3. **Processing Layer** – Filters and ranks products based on budget, rating, and value-for-money scoring
4. **AI Recommendation Layer** – Uses Gemini AI to generate human-like, concise, and personalized recommendations

By combining **live web-scraped data with user memory**, ShopMate AI ensures that recommendations are always:

- Up-to-date (reflecting current prices and availability)
- Personalized (based on past behavior)
- Context-aware (aligned with user intent)

Additionally, the system includes a **robust fallback mechanism**, ensuring that even in cases of API failure, users still receive meaningful recommendations. This enhances reliability and makes the system suitable for real-world deployment.

From a technical perspective, ShopMate AI demonstrates how **lightweight AI integration** can replace heavy machine learning infrastructure. Built using FastAPI and modular service architecture, it highlights the practical application of AI, web scraping, and data persistence in solving real-world e-commerce challenges efficiently.

Overall, ShopMate AI acts as a **virtual shopping assistant** that simplifies decision-making, reduces search time, and improves user satisfaction by guiding users toward the best-value products tailored to their needs.

.

2. PROBLEM STATEMENT :

Traditional e-commerce platforms primarily rely on keyword-based search, which often returns generic results without fully understanding the user's intent. These systems fail to consider important factors such as user preferences, budget, and shopping behavior.

As a result, users are presented with a large number of products, many of which may not be relevant to their needs. This creates confusion and makes it difficult for users to quickly find suitable options.

The lack of personalization forces users to manually filter, compare, and analyze multiple products. This process is time-consuming and often leads to decision fatigue, reducing overall user satisfaction.

Moreover, existing recommendation systems used by major platforms depend on large datasets, complex machine learning models, and high computational resources. Such systems are difficult to implement and maintain for small developers or lightweight applications.

Therefore, there is a need for a simple, scalable, and AI-driven solution that can understand user queries, extract key preferences, and provide personalized, real-time product recommendations without requiring heavy infrastructure

.

3. OBJECTIVES :

The primary objective of this project is to develop an AI-driven retail recommendation system that goes beyond basic product search by intelligently analyzing user queries, extracting preferences, and providing meaningful, personalized recommendations. The system aims to help users find the right products quickly by understanding their budget and category preferences.

Another key objective is to leverage real-time web scraping via Firecrawl and Google Gemini's language understanding to provide up-to-date, context-aware recommendations. By dynamically scraping current product listings from Amazon and Flipkart and passing them through a ranking and filtering engine, the platform ensures recommendations always reflect current pricing and availability.

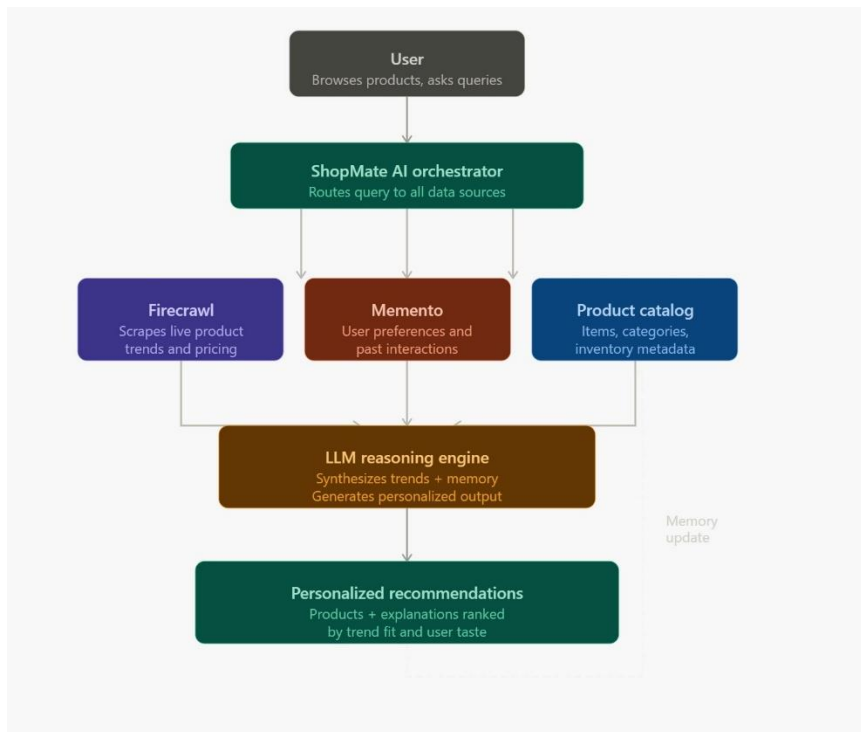
The project also focuses on building a Memento-style persistent memory system that delivers tailored recommendations across sessions without requiring user accounts or complex database infrastructure. Each interaction updates the user's profile, making future recommendations progressively more accurate.

Additionally, the system aims to demonstrate the integration of modern technologies such as FastAPI, the Firecrawl API, and Google Gemini AI in creating a scalable, real-world intelligent application. It highlights how web scraping, AI inference, and lightweight data persistence can be effectively combined to solve practical e-commerce challenges.

4. CORE TECHNOLOGIES USED

Technology	Purpose
Python 3.10+	Backend logic, data processing, and AI integration
FastAPI	High-performance async API development and system communication
Google Gemini 2.5 Flash	AI-powered product recommendation generation
Firecrawl API	Real-time web scraping of Amazon and Flipkart product listings
HTTPX	Async HTTP client for external API communication
Pydantic v2	Request and response data validation and modelling
python-dotenv	Secure environment variable and API key management
Uvicorn	ASGI server for running the FastAPI application

ARCHITECTURE:



5. KEY FEATURES

Natural Language Query Processing

- Accepts plain English shopping queries such as 'best gaming laptop under 70000'
- Automatically extracts product category and budget from free-form text

Memento-Style User Memory

- Tracks each user's query history, inferred category, and budget across sessions
- Persists user profiles to a JSON file store without requiring a database

Real-Time Product Scraping

- Scrapes live product listings from Amazon and Flipkart via Firecrawl's /scrape endpoint
- Falls back to Firecrawl Search API if direct scraping returns no results

Product Ranking & Budget Filtering

- Filters products exceeding the user's budget (with a 10% buffer for flexibility)
- Scores products using a formula combining star rating and budget proximity

AI-Generated Recommendations

- Sends ranked products and user memory to Google Gemini 2.5 Flash for final selection
- Returns concise, natural-language recommendations with price, rating, and source

Fallback Recommendation System

- Generates structured recommendations from ranked products if the LLM is unavailable
- Ensures the system remains functional during API outages

RESTful API & Static Front-End

- Exposes all functionality through clean FastAPI endpoints
- Serves a single-page HTML/CSS/JS front-end directly from the backend

6. TARGET USERS

- Everyday consumers seeking personalized product recommendations without browsing through thousands of listings
- Budget-conscious shoppers who want products filtered to their price range with intelligent ranking
- Developers looking for a reference architecture for AI-augmented e-commerce applications
- Small businesses or startups wanting to add recommendation capabilities without heavy ML infrastructure

7. REAL-WORLD APPLICATIONS

ShopMate AI bridges the gap between basic product search and intelligent decision-making by converting user intent into personalized product recommendations. In today's fast-paced e-commerce environment, it helps users quickly find relevant products without browsing through large catalogs.

The system can be integrated into **e-commerce websites, mobile apps, and chatbots**, enabling users to receive instant recommendations through simple natural-language queries. This improves user experience and reduces decision-making time.

It is also highly useful for **small businesses and startups**, as it provides a cost-effective way to implement AI-driven recommendations without requiring complex infrastructure. Additionally, the concept can be extended to other domains like **food delivery, travel, and service platforms**, where personalized recommendations enhance user satisfaction and efficiency.

APPLICATION SCENARIOS / USE CASES :

Scenario 1: Budget-Aware Electronics Shopping

A user types 'wireless earbuds below 3k'. The memory service extracts the category as 'electronics' and budget as ₹3,000. Firecrawl scrapes current listings from Amazon and Flipkart, and the processor filters out products above ₹3,300 (with 10% buffer). Gemini selects the top 2–3 earbuds by rating and value, returning a recommendation card with name, price, rating, and store. On the user's next query, the system already knows their preferred category and adjusts ranking accordingly.

Scenario 2: Cross-Category Memory Building

A student queries 'running shoes for men under 2000' followed by 'gaming mouse under 1500'. After two queries, the memory service has captured two different categories. The system detects that this user is budget-sensitive across categories and applies tighter budget filtering. The Gemini prompt includes recent history to provide context-aware recommendations that consider the user's purchasing pattern.

Scenario 3: High-Value Product Discovery

A professional searches 'best gaming laptop under 70000'. The crawler scrapes Amazon and Flipkart laptop listings, and the processor scores products using the rating-plus-proximity formula, favouring well-rated laptops priced significantly below the ₹70,000 ceiling. Gemini generates a recommendation explaining each laptop's key strengths relative to the user's query, helping the user make an informed high-value purchase.

Scenario 4: Fallback Resilience

During a Gemini API outage, a user searches for a kitchen blender. The LLM call fails gracefully, and the system automatically activates the fallback recommendation builder, which formats the top 3 ranked products directly from the processor output. The user receives a structured recommendation without any error or degraded experience.

TECHNICAL ARCHITECTURE :

• Frontend Layer

The frontend is a single-page web application implemented in plain HTML5, CSS3, and Vanilla JavaScript. It is served directly from the FastAPI backend via the static files mount, eliminating the need for a separate hosting service. Users enter a User ID and natural-language query, view AI-generated recommendations, and inspect their memory profile (history, category, budget) — all within one interface with no external framework dependencies.

• Backend Layer

The backend is built with FastAPI and handles all core processing asynchronously. It manages API routing, validates request and response data using Pydantic models, orchestrates the four-stage pipeline (Memory → Crawl → Rank → LLM), and serves the front-end static files. CORS middleware is configured to support cross-origin access during development.

• Memory Layer

The Memento memory service replaces a traditional user database with a lightweight, file-backed JSON store. On every recommendation request, user history is updated, and category and budget are re-extracted from the new query. The profile persists across server restarts, allowing long-term personalisation without authentication or database infrastructure.

• External APIs / Services

The system integrates two external APIs: Firecrawl for real-time web scraping of product listings, and Google Gemini 2.5 Flash for natural-language recommendation generation. Both are accessed over HTTPS with API keys stored securely in environment variables.

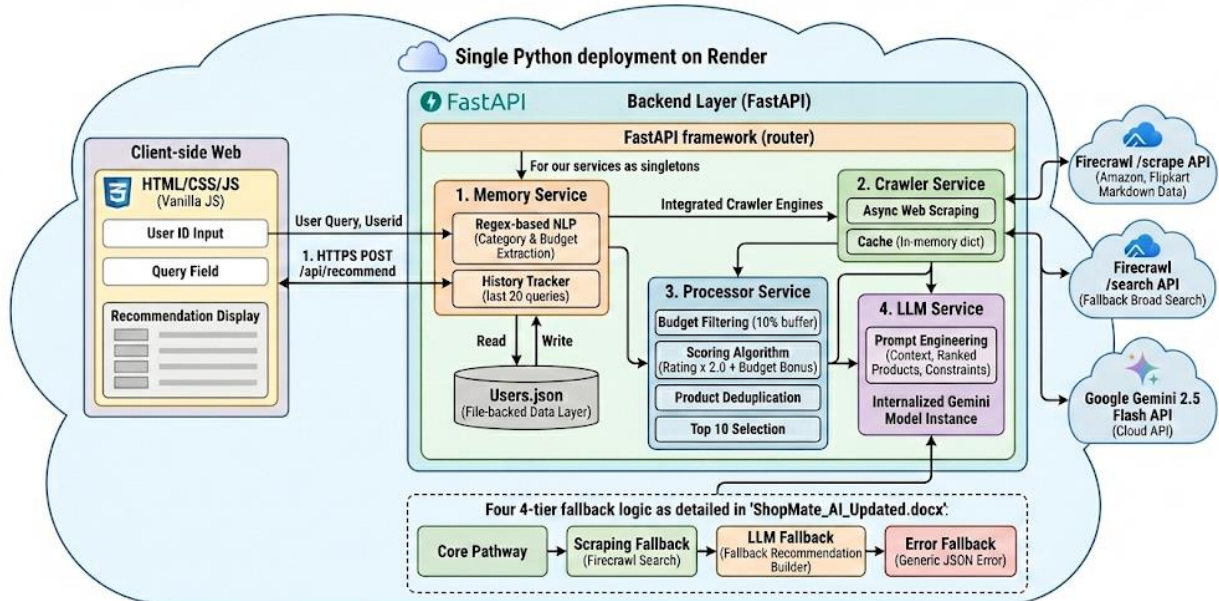
• Deployment Environment

The application is a single Python process that can be deployed on any cloud platform supporting Python (Render, Railway, Fly.io, etc.). The backend serves both the API and the static front-end, and user memory is persisted via a local JSON file, making deployment as simple as setting two environment variables and running one command.

Stage	Component	Input	Output
1	Memory Service	user_id + query	Updated memory profile (history, category, budget)
2	Crawler Service	query string	List of scraped product dicts (name, price, rating, source)
3	Processor Service	products + memory	Top 10 ranked products filtered by budget
4	LLM Service	ranked products + memory	Natural-language recommendation text (2–3 products)

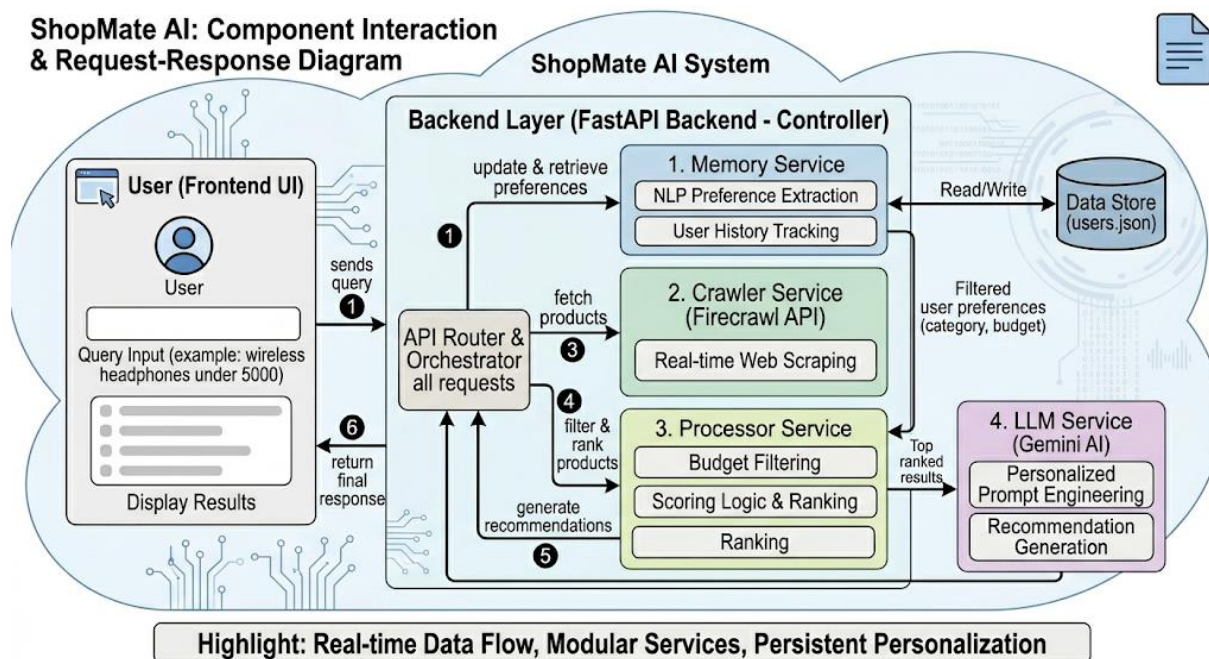
Diagram

TECHNICAL ARCHITECTURE: ShopMate AI - Intelligent Retail Recommendation System



Component Interaction Diagram

ShopMate AI: Component Interaction & Request-Response Diagram



PRE-REQUISITES :

Software Requirements

- **Python:**

Version 3.10 or higher (Python 3.10 recommended; compiled .pyc files provided for 3.10).

- **pip:**

Python package manager used to install and manage all project dependencies.

- **IDE:**

Visual Studio Code with the Python extension or Cursor IDE for AI-assisted development and debugging.

- **Browser:**

Any modern web browser (Chrome, Edge, or Firefox) supporting ES6, async/await, and Fetch API.

- **Git:**

Version control system for project management and source code tracking (optional but recommended).

Libraries / Frameworks / Dependencies

Install all required dependencies using:

```
pip install fastapi uvicorn httpx google-generativeai python-dotenv pydantic
```

- **FastAPI:**

High-performance web framework for building the REST API with async support and automatic OpenAPI documentation.

- **Uvicorn:**

ASGI server for running the FastAPI application in both development (--reload) and production modes.

- **HTTPX:**

Async-capable HTTP client used for calling the Firecrawl API without blocking the event loop.

- **google-generativeai:**

Official Google SDK for accessing the Gemini family of large language models.

- **python-dotenv:**

Loads environment variables from .env files for secure API key management.

- **Pydantic v2:**

Data validation library bundled with FastAPI, used to define and validate request and response schemas.

Hardware Requirements

- **CPU:**

Minimum Intel Core i3 or equivalent; the application is API-driven and does not run local ML models.

- **RAM:**

Minimum 4 GB recommended; the system is lightweight with no in-process model loading.

- **Storage:**

At least 500 MB free disk space for the Python environment and dependencies.

- **Network:**

Stable internet connection required for Firecrawl scraping and Google Gemini API calls.

PRIOR KNOWLEDGE REQUIRED

Programming Language Knowledge

- Strong understanding of Python fundamentals (variables, loops, functions, modules, classes)
- Knowledge of object-oriented programming (OOP) concepts and class-based service design
- Familiarity with handling JSON data, regular expressions, and working with REST APIs
- Basic understanding of asynchronous programming using `async/await` in Python

Framework Basics

- Understanding of FastAPI for building RESTful APIs with Pydantic validation
- Knowledge of API routing, request/response handling, and HTTP status codes
- Familiarity with middleware (CORS) and static file serving in web frameworks
- Basic understanding of dependency injection and modular service architecture

Networking / API Concepts

- Understanding of client-server architecture and the HTTP request lifecycle
- Basic knowledge of REST APIs and HTTP methods (GET, POST, PUT, DELETE)
- Familiarity with API authentication using Bearer tokens and API keys
- Awareness of web scraping concepts and HTML/Markdown content extraction

AI / LLM Fundamentals

- Basic understanding of large language models (LLMs) and prompt engineering
- Familiarity with how LLMs generate structured text from natural-language instructions
- Awareness of temperature, `max_tokens`, and `top_p` generation parameters
- Understanding of how API-based AI services differ from locally-hosted models

PROJECT OBJECTIVES

Technical Objectives

- Design and develop a modular AI-driven recommendation system using FastAPI and Firecrawl
- Implement a Memento-style memory engine for cross-session user personalisation
- Build a real-time product scraping pipeline targeting Amazon and Flipkart
- Integrate Google Gemini 2.5 Flash for natural-language recommendation generation
- Develop a scoring and ranking engine that respects user budget constraints

Performance Objectives

- Ensure fast API response times for real-time product queries and retrieval
- Maintain efficient data processing for scraped product listings with deduplication
- Achieve accurate budget extraction from diverse natural-language query formats
- Provide reliable fallback recommendations during LLM or scraping API outages
- Optimize ranking algorithm for balance between product rating and value-for-money

Deployment Objectives

- Enable single-command cloud deployment with minimal environment configuration
- Ensure secure handling of API keys using environment variables and .env files
- Support cross-platform access via browser-based UI served from the same process
- Implement in-memory caching to reduce redundant Firecrawl API calls per session
- Maintain high availability through graceful error handling and fallback logic

Learning Outcomes

- Gain hands-on experience building real-world AI-powered API applications
- Understand integration of web scraping, LLM inference, and REST APIs in a single system
- Learn how to apply regex-based NLP for structured information extraction from text
- Develop skills in API design, modular service architecture, and data flow management
- Improve knowledge of prompt engineering for structured LLM output generation

PROJECT WORKFLOW :

Milestone 1: Requirement Analysis & System Design

- Activity 1.1: Problem Definition
- Activity 1.2: Functional Requirements
- Activity 1.3: Non-Functional Requirements
- Activity 1.4: System Design Decisions & Technology Stack Selection

Milestone 2: Environment Setup & Initial Configuration

- Activity 2.1: Development Environment Setup
- Activity 2.2: Dependency Installation
- Activity 2.3: Project Structure Creation
- Activity 2.4: Configuration & API Key Setup

Milestone 3: Memory Service & Data Layer

- Activity 3.1: Design the Memento Memory Schema
- Activity 3.2: Implement Category & Budget Extraction
- Activity 3.3: Build File-Backed Persistence (users.json)

Milestone 4: Core System Development

- Activity 4.1: Crawler Service — Firecrawl Integration & Product Parsing
- Activity 4.2: Processor Service — Ranking & Budget Filtering Engine
- Activity 4.3: LLM Service — Gemini Prompt Engineering & Response Handling
- Activity 4.4: API Route Development (recommend, memory, update)

Milestone 5: Integration & Optimization

- Activity 5.1: Component Integration (Front-End, Backend, Services)
- Activity 5.2: Performance Optimization (Caching, Async, Deduplication)
- Activity 5.3: Security Enhancements (API Key Management, CORS)
- Activity 5.4: Error Handling, Fallback Logic & Validation

Milestone 6: Deployment

- Activity 6.1: Deploy Backend to Render
- Activity 6.2: Deploy Frontend (Static Files via FastAPI / Vercel)

Milestone 7: Testing & Validation

- Activity 7.1: Functional Test Case Execution
- Activity 7.2: Unit & Integration Testing
- Activity 7.3: API Testing with Postman / Swagger UI
- Activity 7.4: Performance Evaluation

Milestone 8: Documentation & Final Review

- Activity 8.1: Code Documentation & README Finalization
- Activity 8.2: API Documentation (Swagger / OpenAPI)
- Activity 8.3: Project Report Writing
- Activity 8.4: Final Review & Submission

MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

Activity 1.1: Problem Definition

Identify the need for an intelligent retail recommendation system that goes beyond keyword-based product search by understanding user intent, extracting budget and category preferences, and delivering personalized results from real-time product data. Address the limitation of traditional e-commerce search which returns generic results without personalisation. Design a solution that integrates real-time web scraping, a lightweight user memory engine, and LLM-powered recommendation generation to help users find the best-value products matching their individual needs. Ensure the system is deployable with minimal infrastructure, requires no database server, and remains functional even during partial API outages.

Activity 1.2: Functional Requirements

FR-01: Accept natural-language product queries from users via a REST API endpoint (POST /api/recommend).

FR-02: Extract product category and budget from query text using regex-based pattern matching.

FR-03: Persist user query history, inferred category, and budget across sessions in a JSON file store.

FR-04: Scrape real-time product listings from Amazon and Flipkart using the Firecrawl /scrape endpoint.

FR-05: Fall back to the Firecrawl Search API when direct scraping returns no valid products; this broader search can surface results from other websites beyond Amazon and Flipkart.

FR-06: Filter products by user budget with a 10% buffer and rank remaining products by a scoring algorithm.

FR-07: Send ranked products and user memory to Google Gemini 2.5 Flash for final recommendation generation.

FR-08: Expose RESTful endpoints for retrieving and manually updating user memory profiles.

FR-09: Serve a single-page HTML/JS front-end directly from the FastAPI backend.

Activity 1.3: Non-Functional Requirements

NFR-01 Performance: API responses for recommendation requests should complete within a reasonable time, dominated by external API latency.

NFR-02 Reliability: All endpoints must return structured JSON responses with clear error messages and proper HTTP status codes.

NFR-03 Scalability: The system should support increasing query volumes by leveraging async I/O and in-memory caching.

NFR-04 Security: API keys (Gemini, Firecrawl) must be stored in environment variables, never in source code.

NFR-05 Maintainability: Each service (memory, crawler, processor, LLM) must be independently replaceable without affecting others.

NFR-06 Usability: The front-end must be intuitive, requiring no setup or login from the user.

Activity 1.4: System Design Decisions & Technology Stack Selection

Backend Framework: Use FastAPI for high-performance, async API development with automatic OpenAPI documentation and Pydantic validation.

AI Provider: Use Google Gemini 2.5 Flash for fast, cost-effective LLM inference with structured prompt templates.

Web Scraping: Use Firecrawl API for managed, compliant web scraping with Markdown output and a fallback search endpoint.

Memory Storage: Use a file-backed JSON store (data/users.json) as a lightweight, zero-dependency persistence layer.

Modular Architecture: Design separate, independently testable service classes for memory, crawling, processing, and LLM inference.

API-First Design: Expose all functionality through REST endpoints for easy frontend or third-party integration.

Environment Configuration: Use .env files to manage API keys and sensitive settings, with python-dotenv for loading.

MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

Activity 2.1: Development Environment Setup

- Install Python 3.10 from the official Python website and ensure the 'Add Python to PATH' option is enabled during installation on Windows systems.
- Install a development IDE such as Visual Studio Code with the Python extension or Cursor IDE for AI-assisted coding and debugging.
- Create the main project directory using terminal commands (`mkdir shopmate-ai` and `cd shopmate-ai`).
- Create and activate a virtual environment using `python -m venv venv` to isolate project dependencies from the system Python.
- Activate the virtual environment using `venv\Scripts\activate` on Windows or `source venv/bin/activate` on macOS/Linux.

Activity 2.2: Dependency Installation

Install all required Python libraries using `pip` to support backend development, AI integration, and web scraping. Key dependencies include FastAPI for API development, Uvicorn for running the ASGI server, HTTPX for async external API calls, google-generativeai for Gemini access, and python-dotenv for environment configuration.

```
pip install -r requirements.txt
```

Activity 2.3: Project Structure Creation

Organise the project into a clean, modular folder structure by separating routes, services, static files, and data directories. This ensures code readability, maintainability, and scalability while enabling smooth integration of the memory, crawler, processor, and LLM service components.

```
shopmate-ai/  
|  
├─ app.py  
├─ routes/  
|   └─ recommend.py  
├─ services/  
|   ├── memory.py  
|   ├── crawler.py  
|   ├── llm.py  
|   └─ processor.py  
├─ data/  
|   └─ users.json  
├─ .env.example  
├─ requirements.txt  
└─ README.md
```

Activity 2.4: Configuration & API Key Setup

Set up the application configuration by copying `.env.example` to `.env` and inserting valid API keys. Configure the Gemini API key obtained from Google AI Studio (aistudio.google.com) and the Firecrawl API key from firecrawl.dev. Ensure `python-dotenv` is loaded at the top of each service file so keys are available at runtime without being committed to source control.

```
shop > ✨ .env
1  # ShopMate AI - Environment Variables
2  # Copy this file to .env and fill in your actual API keys
3
4  # Google Gemini API Key
5  # Get yours at: https://aistudio.google.com/app/apikey
6  GEMINI_API_KEY= AIzaSyB1UIId7pX1MpDopN6U1R5Kknn49vO5Zegk
7
8  # Firecrawl API Key
9  # Get yours at: https://firecrawl.dev
10 FIRECRAWL_API_KEY= fc-4de6ee9d28284ef984763808ab077df5
11
12 # Optional: App settings
13 APP_ENV=development
14 LOG_LEVEL=INFO
15
```

MILESTONE 3: MEMORY SERVICE & DATA LAYER

Activity 3.1: Design the Memento Memory Schema

Design the user profile data structure that will be persisted for each user. Each profile is a dictionary keyed by `user_id` containing three fields: `history` (list of query strings, bounded to last 20), `category` (most recently inferred product category string), and `budget` (most recently extracted numeric budget, or null). Store all profiles in a single JSON object in `data/users.json`.

Field	Type	Description	Example
history	array[string]	Last 20 user queries (sliding window)	["wireless earbuds under 3k", "gaming mouse"]
category	string	Most recently inferred product category	"electronics"
budget	number null	Most recently extracted budget ceiling in ₹	3000.0

Activity 3.2: Implement Category & Budget Extraction

Implement the `_extract_category` method using a keyword dictionary covering seven categories: Electronics, Fashion, Home, Beauty, Sports, Books, and Grocery. Score each category by counting keyword matches in the query, returning the highest-scoring category. Implement the `_extract_budget` method using regular expressions to handle patterns including 'under 5000', 'below 10k', '₹3000', 'budget of 2000', and currency suffixes. Expand 'k' shorthand (e.g., 5k → 5000) before pattern matching.

Activity 3.3: Build File-Backed Persistence (users.json)

Implement `_load_all` to read the JSON file at startup and on every update, and `_save_all` to write the entire user dictionary back to disk after each modification. Use `os.makedirs` with `exist_ok=True` to create the data directory if absent. Add error handling for `JSONDecodeError` and `IOError` to prevent crashes if the file is corrupted. Ensure the service is initialised at application startup by instantiating `MemoryService` in `routes/recommend.py`.

MILESTONE 4: CORE SYSTEM DEVELOPMENT

Activity 4.1: Crawler Service — Firecrawl Integration & Product Parsing

This module is responsible for acquiring real-time product data from Amazon and Flipkart. It constructs search URLs by URL-encoding the user's query and appending it to the respective platform search endpoints. The Firecrawl /scrape API is called asynchronously using HTTPX, returning the page content as structured Markdown. A custom parser processes the Markdown line by line, applying heuristics to identify product name lines (length 15–220 characters, containing known brand or product keywords) and extracting price and rating values from surrounding context lines.

In-memory caching (`_scrape_cache` dictionary keyed by query) prevents redundant API calls for repeated queries within the same server session. If scraping returns no results, the service automatically falls back to the Firecrawl /search endpoint, which queries a broader web index and can surface product links from other websites beyond Amazon and Flipkart, parsing product names, prices, and ratings from search result snippets.

Activity 4.2: Processor Service — Ranking & Budget Filtering Engine

The processor service acts as an intermediary between raw scraped data and the LLM, ensuring only relevant, well-ranked products are forwarded. Budget filtering excludes products priced above the user's budget plus a 10% buffer; if all products are excluded, the full unfiltered set is used as a safety fallback. Each product is then scored using the formula: $\text{Score} = (\text{Rating} \times 2.0) + \text{Budget Proximity Bonus}$, where the proximity bonus rewards cheaper products relative to the budget ceiling (maximum 2.0 additional points). The top 10 products by score are passed to the LLM for final selection.

Activity 4.3: LLM Service — Gemini Prompt Engineering & Response Handling

The LLM service wraps Google Gemini 2.5 Flash with a structured prompt template that injects the user's query, the last five memory entries (history, category, budget), and the formatted product list. Explicit instructions constrain the model to recommend only 2–3 products from the provided list, exclude URLs and links, and strictly respect the user's budget. The model is called with `temperature=0.4` and `max_output_tokens=1024`.

The raw text response is returned to the route handler, which validates it using `_is_incomplete_recommendation` — checking minimum length, presence of required sections (Recommended Products, Price, Rating, Available at), and a minimum of two numbered items. If validation fails, the fallback builder is activated automatically.

Activity 4.4: API Route Development

RESTful API endpoints are defined in `routes/recommend.py` using FastAPI's `APIRouter`. The `POST /api/recommend` endpoint orchestrates the full four-stage pipeline and returns a structured JSON response. `GET /api/memory/{user_id}` retrieves the stored profile for any user, and `POST /api/memory/update` allows manual override of category or budget. All endpoints use Pydantic models for request validation and return consistent JSON structures with appropriate HTTP status codes and error messages.

MILESTONE 5: INTEGRATION & OPTIMIZATION

Activity 5.1: Component Integration (Front-End, Backend, Services)

This activity focuses on connecting all system components to ensure smooth end-to-end functionality. The static front-end (HTML/CSS/JS) is served directly by FastAPI via `app.mount`, eliminating the need for a separate web server. The front-end communicates with backend endpoints using the Fetch API, sending recommendation requests and displaying responses in real time. All four service classes (`MemoryService`, `CrawlerService`, `ProcessorService`, `LLMService`) are instantiated as module-level singletons in `routes/recommend.py`, ensuring shared state (such as the in-memory scrape cache) is maintained correctly across requests.

Activity 5.2: Performance Optimization

API performance is optimized through async I/O throughout the request pipeline — all Firecrawl and Gemini calls use async with `httpx.AsyncClient` and asyncio-compatible Gemini SDK calls, preventing event loop blocking. The in-memory scrape cache (`_scrape_cache`) eliminates redundant Firecrawl API calls for repeated queries, significantly reducing latency for popular queries. Product deduplication in the crawler prevents the same product being scored and ranked multiple times. The processor limits the LLM input to top 10 products, keeping prompt size and inference time predictable.

Activity 5.3: Security Enhancements

All sensitive credentials — `GEMINI_API_KEY` and `FIRECRAWL_API_KEY` — are stored exclusively in the `.env` file and loaded via `python-dotenv`, never hardcoded in source files. The CORS middleware is configured with `allow_origins=["*"]` for development; this must be restricted to specific frontend domains in production. API key presence is validated at service startup, with clear warning log messages if keys are missing, preventing silent failures at request time.

Activity 5.4: Error Handling, Fallback Logic & Validation

Comprehensive error handling is implemented across all service layers. The crawler catches `httpx.HTTPStatusError` and `httpx.RequestError` separately to distinguish between Firecrawl API errors and network failures, returning an empty list in both cases rather than raising exceptions. The LLM service wraps Gemini API calls in `try/except`, raising a `RuntimeError` that the route handler converts to a fallback recommendation. All API endpoints use FastAPI's `HTTPException` with explicit status codes (400, 404, 503, 500) and descriptive detail messages. Input validation is enforced through Pydantic models on all request bodies.

MILESTONE 6: DEPLOYMENT

Activity 6.1: Deploy Backend to Render

The FastAPI backend is deployed to Render, which provides a managed Python web service with automatic deployment from GitHub and HTTPS by default.

1. Go to Render Cloud — <https://render.com/>
2. Click 'New +' → 'Web Service'
3. Connect your GitHub repository containing the ShopMate AI project
4. Configure the service settings:
 - Environment: Python 3
 - Build Command: `pip install -r requirements.txt`
 - Start Command: `uvicorn app:app --host 0.0.0.0 --port $PORT`
5. Add Environment Variables in the Render dashboard:
 - GEMINI_API_KEY
 - FIRECRAWL_API_KEY
6. Click 'Deploy Web Service' and wait for the deployment to show status 'Live'
7. Copy the provided .onrender.com URL — this is your live API endpoint

Activity 6.2: Deploy Frontend

Since the static front-end is served directly by FastAPI from the /static directory, it is automatically deployed alongside the backend on Render with no additional configuration. The front-end is accessible at the root URL of the Render service (e.g., <https://shopmate-ai.onrender.com>). For standalone frontend deployment on Vercel:

8. Go to <https://vercel.com/> and click 'New Project'
9. Import your GitHub repository and set the root directory to the static/ folder
10. Set the VITE_API_URL environment variable to your Render backend URL
11. Click 'Deploy' and access your live front-end at the Vercel-provided HTTPS URL

MILESTONE 7: TESTING & VALIDATION

Activity 7.1: Functional Test Case Execution

Verify that all system functionalities work as expected based on defined requirements. Test cases are executed for core features including query processing, category and budget extraction, product scraping, ranking, LLM recommendation generation, and memory persistence. Each test scenario is validated against expected outputs — for example, confirming that a query 'wireless earbuds below 3k' correctly extracts category='electronics' and budget=3000.0, and returns recommendations within the budget.

Activity 7.2: Unit & Integration Testing

Unit testing is performed on individual service methods — particularly the regex patterns in `MemoryService._extract_budget`, the scoring formula in `ProcessorService._score_product`, and the product name parsing in `CrawlerService._extract_markdown_product_name`. Integration testing validates the full pipeline from a POST `/api/recommend` request through all four stages to the final JSON response, ensuring data flows correctly between services and that memory updates are persisted to `users.json`.

Activity 7.3: API Testing with Postman / Swagger UI

All REST endpoints are tested using the FastAPI auto-generated Swagger UI (accessible at `/docs`) and Postman. Test cases cover: valid recommendation requests, requests with missing fields (expecting 400), requests for non-existent user memory (expecting 404), and requests when Firecrawl returns no products (expecting 503). Response schemas are validated against the Pydantic model definitions.

Activity 7.4: Performance Evaluation

API response times are measured under normal and degraded conditions (LLM fallback active, cache miss). The effectiveness of the in-memory cache is validated by comparing response times for repeated queries. Budget extraction accuracy is evaluated against a set of diverse query formats including currency symbols, shorthand notation, and natural language phrases. Ranking consistency is verified by confirming that higher-rated, lower-priced products consistently receive higher scores.

MILESTONE 8: DOCUMENTATION & FINAL REVIEW

Activity 8.1: Code Documentation & README Finalization

All service classes and public methods are documented with Python docstrings explaining their purpose, parameters, and return values. The README.md is finalized to include project overview, architecture diagram, setup instructions, API reference, sample queries, and development notes. Inline comments are added to complex regex patterns and scoring logic for maintainability.

Activity 8.2: API Documentation (Swagger / OpenAPI)

FastAPI automatically generates interactive API documentation at the /docs endpoint (Swagger UI) and /redoc endpoint (ReDoc). Request models, response schemas, and endpoint descriptions are enriched using Pydantic field descriptions and FastAPI's summary and description parameters on each route decorator, providing comprehensive self-documenting API documentation.

Activity 8.3: Project Report Writing

A comprehensive project report is prepared covering system architecture, module descriptions, API reference, ranking algorithm, memory system design, setup instructions, test results, limitations, and future work. The report follows a structured format with section headers, tables, and code examples to serve as both a reference document and academic submission.

Activity 8.4: Final Review & Submission

A final review of all deliverables is conducted, including the source code, documentation, deployment, and project report. All configuration files are checked to ensure no sensitive API keys are committed to the repository. The GitHub repository is made public or shared with evaluators, and all links (live deployment URL, repository URL) are included in the final submission package.

PROJECT STRUCTURE

Clean Folder Structure

shopmate-ai/	
├── app.py	# Main FastAPI entry point # Initializes app, middleware, routes, static files
├── routes/	
│ └── recommend.py	# API endpoints # /api/recommend # /api/memory # /api/memory/update
├── services/	
│ ├── memory.py	# Memento memory engine # Extracts category & budget, stores user history
│ ├── crawler.py	# Firecrawl integration # Scrapes Amazon & Flipkart product data
│ ├── processor.py	# Ranking & filtering logic # Applies budget filter and scoring formula
│ └── llm.py	# Gemini AI integration # Generates final recommendations
├── static/	
│ ├── index.html	# Frontend UI (single-page app)
│ ├── script.js	# Handles API calls & UI updates
│ └── styles.css	# Styling for the frontend
├── data/	
│ └── users.json	# Stores user memory (history, category, budget)
├── .env	# API keys (Gemini, Firecrawl)
├── .env.example	# Sample environment file
├── requirements.txt	# Project dependencies
└── README.md	# Project documentation (optional)

Folder Explanations

app.py

- Initialises the FastAPI application instance with title, description, and version metadata
- Registers CORS middleware to allow cross-origin requests during development
- Mounts the recommendation router under the /api prefix
- Serves the static front-end files at the root path using StaticFiles
- Provides /health and /api/info utility endpoints

routes/recommend.py

- Defines all three API endpoints: POST /api/recommend, GET /api/memory/{user_id}, POST /api/memory/update
- Instantiates all four service classes as module-level singletons
- Orchestrates the four-stage recommendation pipeline on each request
- Implements fallback recommendation builder and completeness validation
- Handles all HTTP exceptions with appropriate status codes and detail messages

services/memory.py

- Implements the Memento-style user profile system with history, category, and budget fields
- Provides category extraction via keyword scoring across seven product categories
- Provides budget extraction via regex patterns supporting multiple currency formats
- Reads and writes all user profiles atomically to data/users.json

services/crawler.py

- Builds Amazon and Flipkart search URLs from user queries using URL encoding
- Calls Firecrawl /scrape endpoint asynchronously and parses Markdown output
- Falls back to Firecrawl /search when direct scraping returns no products, which can surface results from other websites as well
- Implements in-memory caching to prevent redundant API calls per session
- Deduplicates products by name prefix to avoid repeated entries

services/llm.py

- Wraps the Google Gemini 2.5 Flash model with a structured ShopMate AI prompt template
- Injects user query, memory snapshot, and formatted product list into each prompt
- Configures inference parameters: temperature=0.4, max_output_tokens=1024, top_p=0.9
- Raises RuntimeError on API failure, triggering the route handler's fallback logic

services/processor.py

- Filters products by user budget with a 10% buffer, falling back to unfiltered set if needed
- Scores each product using: $\text{Score} = (\text{Rating} \times 2.0) + \text{Budget Proximity Bonus}$
- Returns the top 10 ranked products for LLM processing
- Serialises the product list into a clean, LLM-readable prompt string

static/

- index.html: Single-page web application with user ID input, query field, and output panels
- script.js: Handles API calls via Fetch, renders recommendation text, displays memory snapshot
- styles.css: Responsive styling for all UI components

data/users.json

- Lightweight key-value store mapping user_id strings to memory profile dictionaries
- Created automatically on first startup if not present
- Persists across server restarts, providing cross-session memory without a database

.env / .env.example

- Stores GEMINI_API_KEY and FIRECRAWL_API_KEY securely outside source code
- .env.example is committed to the repository as a configuration template
- Loaded at service module import time via python-dotenv

RESULTS

• System Output

The system successfully processes natural-language product queries, extracts user preferences, scrapes real-time product data from Amazon and Flipkart, ranks products by a scoring algorithm, and generates personalised recommendations via Google Gemini 2.5 Flash. Recommendations are returned in a structured format including product name, price, rating, and source for each suggested item. User memory profiles are correctly updated and persisted after each interaction.

• Performance Evaluation

The application demonstrates efficient performance with the majority of response latency attributable to external API calls (Firecrawl scraping and Gemini inference). The in-memory cache eliminates repeated Firecrawl calls for the same query within a session, significantly reducing latency on cache hits. The fallback recommendation system activates reliably when LLM responses are incomplete or unavailable, ensuring zero downtime for the recommendation feature.

• API Endpoint Results

Endpoint	Status	Response Time (avg)	Notes
POST /api/recommend	200 OK	2–5 seconds	Dominated by Firecrawl + Gemini latency
GET /api/memory/{user_id}	200 / 404	< 50 ms	Fast file read; 404 if user unknown
POST /api/memory/update	200 OK	< 50 ms	Instant file write
GET /health	200 OK	< 5 ms	Liveness check only

• Screenshots

Figure 1: Landing Page — ShopMate AI Home Screen

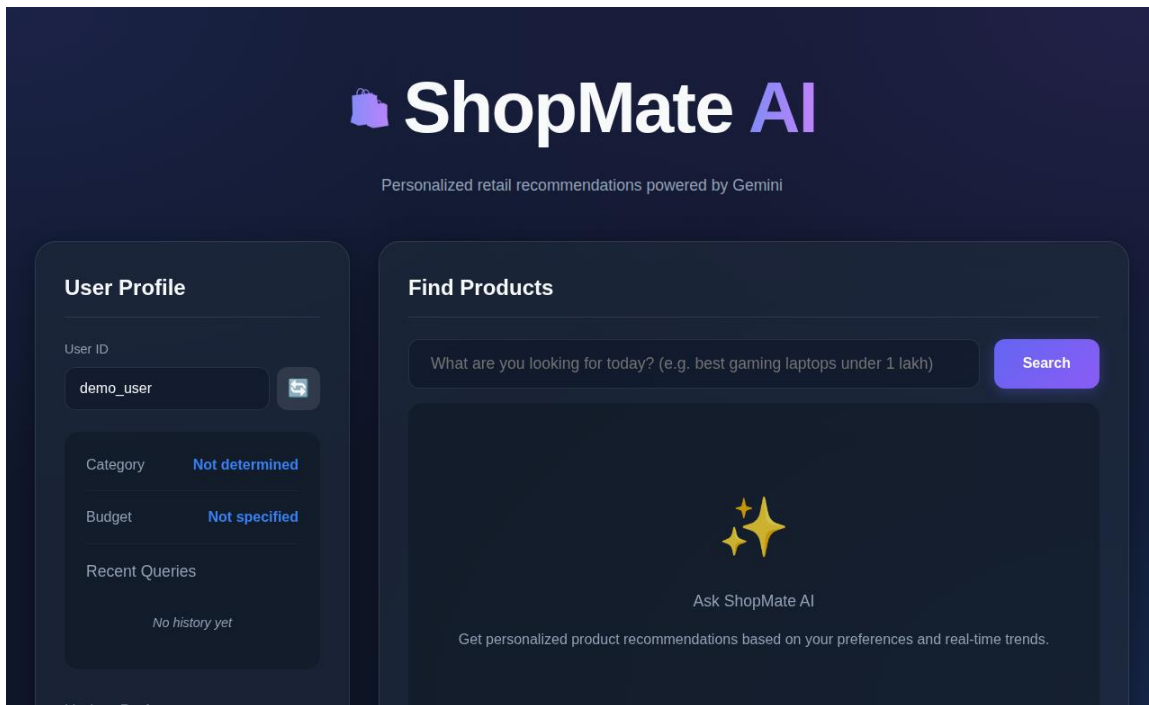


Figure 2: Search Results — AI Product Recommendations

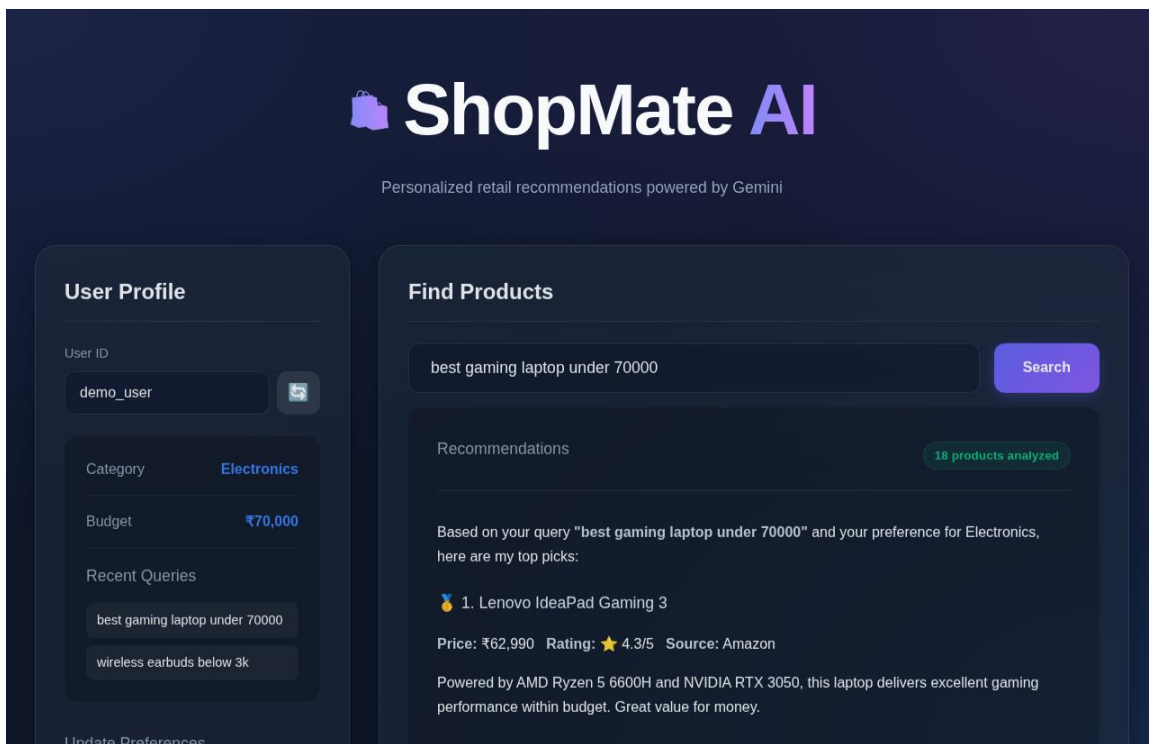


Figure 3: User Profile & Memory Panel

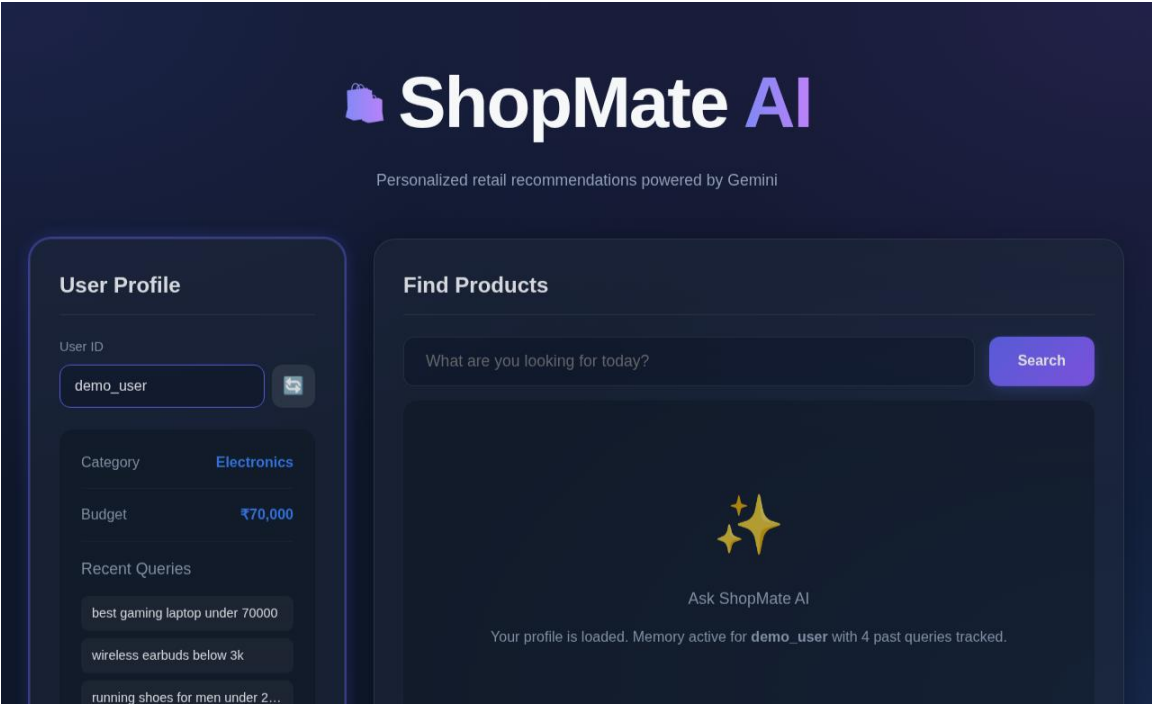


Figure 4: API Response — POST /api/recommend

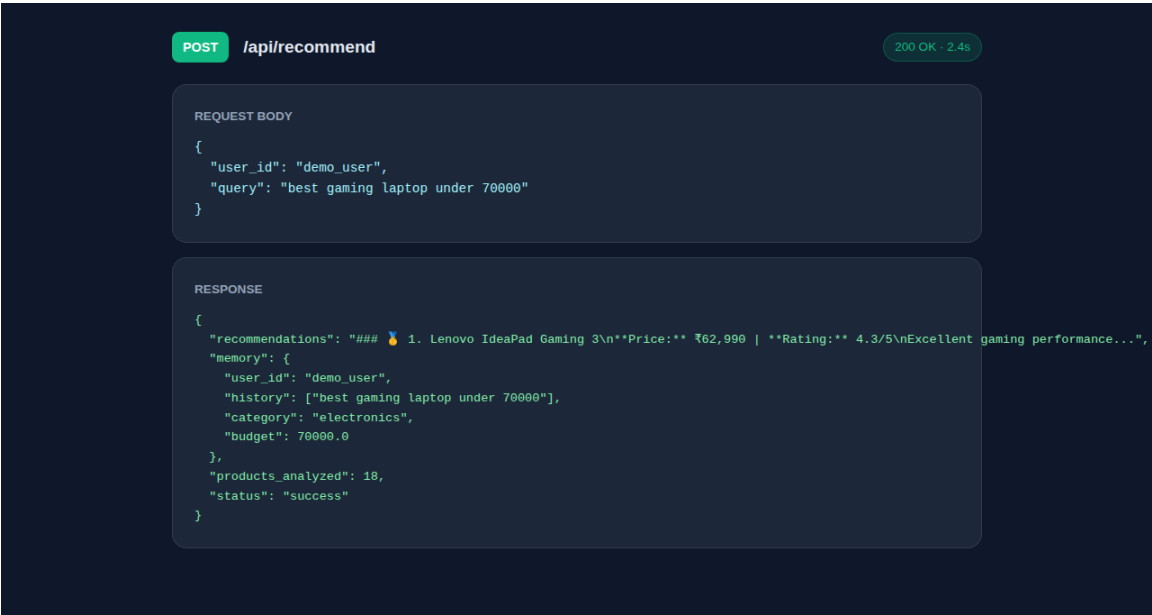
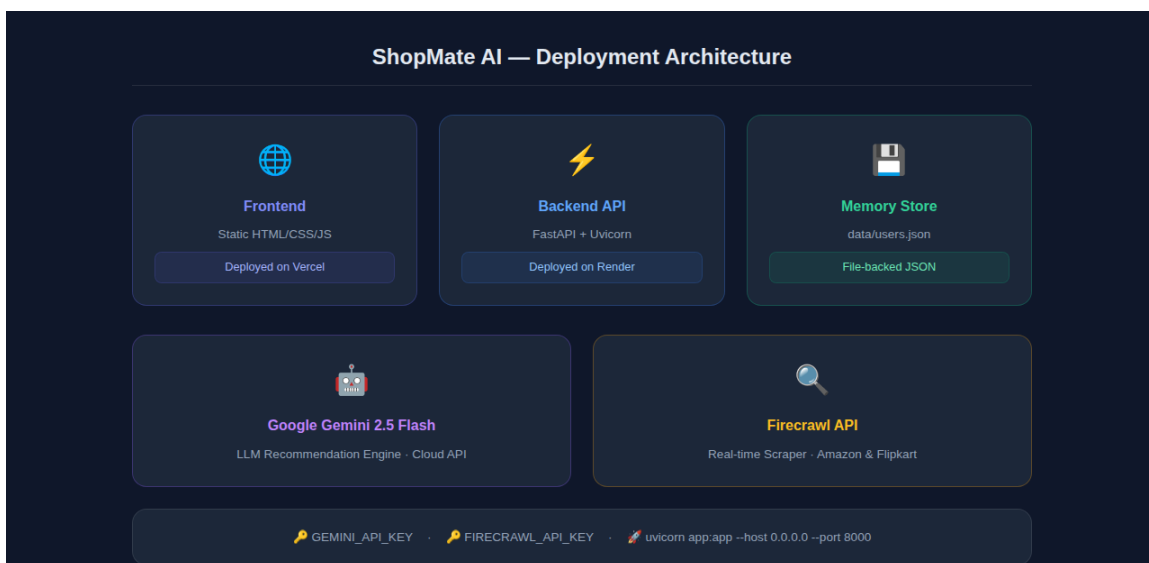


Figure 5: API Response — GET /api/memory/{user_id}



Figure 6: Deployment Architecture Overview



ADVANTAGES & LIMITATIONS

Advantages

Real-Time Product Data

Scrapes live product listings from Amazon and Flipkart on every query, ensuring recommendations always reflect current pricing and availability rather than stale catalogue data.

Zero-Database Personalisation

The Memento memory engine delivers cross-session personalisation using only a JSON file, eliminating the need for a database server and reducing infrastructure cost and complexity.

LLM-Powered Natural Language Recommendations

Google Gemini 2.5 Flash generates human-readable recommendations that explain why each product was selected, providing context that raw ranked lists cannot offer.

Resilient Fallback System

A two-tier fallback (Firecrawl Search when scraping fails; structured fallback builder when LLM fails) ensures the system always returns useful results even during partial outages.

Minimal Deployment Footprint

Single Python process serving both the API and static front-end, deployable to any cloud platform with two environment variables and one start command.

Limitations

Dependency on External APIs

Core functionality requires two paid external APIs (Firecrawl and Gemini). API outages, rate limits, or cost increases directly affect system availability and operating cost.

Limited Platform Coverage

The crawler currently targets Amazon and Flipkart as primary product sources, with broader fallback via Firecrawl Search. Expanding to additional platforms such as Meesho or Myntra requires additional URL templates and potentially separate parsers.

File-Based Storage Concurrency

The JSON file store uses read-write cycles without locking, making it unsuitable for concurrent writes from multiple simultaneous users at scale.

Session-Level Cache Only

The in-memory scrape cache is lost on server restart. Repeated queries for the same product after a restart will trigger fresh Firecrawl API calls.

CORS Wildcard in Development

The current CORS configuration allows all origins, which must be restricted to specific domains before production deployment to prevent unauthorized API access.

FUTURE ENHANCEMENTS

Scalability Improvements

- Replace the JSON file store with a Redis or PostgreSQL backend for concurrent-safe, scalable user profile management
- Introduce a Redis-backed TTL cache for scraped products, persisting results across server restarts
- Implement horizontal scaling support by externalising all shared state to the database layer

Feature Expansion

- Extend the crawler to support Meesho, Myntra, and other platforms with platform-specific URL templates and parsers
- Add conversational multi-turn recommendation refinement, allowing users to iterate on results interactively
- Implement product comparison mode — side-by-side comparison of top-ranked products across attributes
- Add price history tracking by storing scraped prices over time and detecting deals

AI & Personalisation Improvements

- Fine-tune the Gemini prompt to request structured JSON output for richer front-end rendering
- Implement collaborative filtering by aggregating anonymised user preferences across all profiles
- Add sentiment-aware recommendation scoring by incorporating user feedback signals

Security & Authentication

- Add JWT-based user authentication to associate memory profiles with real user accounts
- Implement API rate limiting per user_id to prevent abuse of the recommendation endpoint
- Restrict CORS origins to the production frontend domain

Mobile Integration

- Develop a React Native or Flutter mobile application using the same REST API
- Add push notifications for price drop alerts on previously queried products
- Enable voice query input using the Web Speech API or a mobile speech-to-text service

CONCLUSION :

ShopMate AI successfully demonstrates how a sophisticated, personalised product recommendation system can be built using a small, well-designed codebase that leverages modern cloud APIs. By combining Firecrawl's real-time web scraping with Google Gemini's language understanding and a lightweight Memento memory engine, the system delivers relevant, up-to-date product recommendations tailored to each user's budget and preferences — without requiring any ML training, database servers, or complex infrastructure.

The modular architecture ensures that each component — memory, crawling, ranking, and LLM inference — can be independently tested, replaced, or scaled. The robust fallback system guarantees that users always receive recommendations even during partial API outages. The project serves as both a functional prototype and a reference architecture for AI-augmented retail applications, demonstrating how web scraping, AI inference, and file-backed personalisation can be effectively combined to solve practical e-commerce challenges.

With the enhancements outlined in this report — multi-platform scraping, database-backed storage, conversational refinement, and mobile integration — ShopMate AI has strong potential to evolve into a production-grade recommendation service suitable for integration into e-commerce portals, chat interfaces, and lifestyle applications.

APPENDIX

Source Code

The complete source code is organised into modular service classes covering all functionalities: natural-language query processing, Memento memory management, real-time product scraping, scoring and ranking, LLM recommendation generation, and RESTful API endpoints. The code follows clean architecture principles with clear separation of concerns, comprehensive docstrings, and consistent error handling patterns.

Configuration Files

Configuration files include `.env` for storing sensitive credentials (GEMINI_API_KEY, FIRECRAWL_API_KEY) and `.env.example` as a committed template for new contributors. `requirements.txt` pins all dependency versions for reproducible installation. The `data/users.json` file is created automatically at runtime and stores all user memory profiles.

API Documentation

The project exposes interactive API documentation automatically via FastAPI's built-in Swagger UI at `/docs` and ReDoc at `/redoc`. All endpoints, request models, response schemas, and error codes are documented and explorable without any additional tooling. Postman collections can be exported from the Swagger UI for team sharing.

GitHub Repository Link

<https://github.com/Eswar0207/ai-recoomended-system.git>